

Simulation d'Érosion Hydraulique et Thermique en Temps Réel sur GPU via Compute Shaders

César CHARBEY, Thomas BARASCUD, Jérémy NICOLAS, Lucas PAULO
Université de Montpellier - Faculté des Sciences
Projet de TER - Encadrant : Nicolas Lutz

Année Universitaire 2025-2026

Résumé

La génération procédurale de terrains virtuels réalistes nécessite la simulation de phénomènes physiques complexes s'étalant sur des temps géologiques. Ce document présente la conception et l'implémentation sur carte graphique (GPU), via la technologie des Compute Shaders, du modèle d'érosion hydraulique rapide initialement proposé par Mei et al. [1]. Afin de pallier le manque de détails géométriques inhérent à ces approches, tel que mis en évidence par Schott et al. [3], nous proposons et intégrons une extension simulant l'érosion thermique (éboulements gravitationnels). Nos résultats démontrent qu'il est possible d'augmenter drastiquement le réalisme des pentes abruptes et la distribution sédimentaire, tout en maintenant des performances d'exécution colossales (avoisinant les 1000 FPS), prouvant ainsi la viabilité de ce modèle étendu pour des applications interactives exigeantes.

1 Introduction

Dans le domaine de l'informatique graphique et de la création de mondes virtuels, la modélisation topographique crédible est un enjeu majeur. L'apparence organique des chaînes de montagnes, des lits de rivières et des vallées est le résultat de millénaires d'interactions entre les précipitations atmosphériques et la croûte terrestre. Les mouvements d'eau sculptent ces terrains par des processus couplés de dissolution mécanique, de transport hydrodynamique de matière en suspension, et de sédimentation.



FIGURE 1 – L'érosion hydraulique et thermique façonnent naturellement la géomorphologie des paysages, creusant des vallées et formant des éboulis.

Si les méthodes basées sur la mécanique des fluides offrent une fidélité physique incontestable pour reproduire ces structures, elles ont historiquement souffert de temps de calcul énorme, les reléguant au rendu hors-ligne (effets spéciaux pour le cinéma, simulations géologiques). Ce projet vise à concevoir une solution d'érosion en temps réel en s'appuyant sur l'accélération matérielle des processeurs graphiques (GPU), sur la base des travaux de Mei et al. [1]. De surcroît, pour répondre aux limites de précision de ce modèle classique, nous avons implémenté un système dynamique d'érosion thermique, garantissant l'effondrement physiquement réaliste des falaises trop abruptes laissées par le passage de l'eau.

2 État de l'Art et Positionnement

La génération de terrains s'est d'abord appuyée sur des approches purement mathématiques, basées sur la superposition de bruits pseudo-aléatoires (bruit de Perlin, Simplex, ou fractales). Si ces algorithmes garantissent une exécution quasi-instantanée et une empreinte mémoire minimale, ils souffrent d'une lacune conceptuelle majeure : un manque inhérent de cohérence géologique. Les paysages ainsi générés sont uniformément vallonnés et fondamentalement dénués de réseaux de drainage, de crêtes acérées ou de dépôt sédimentaire.

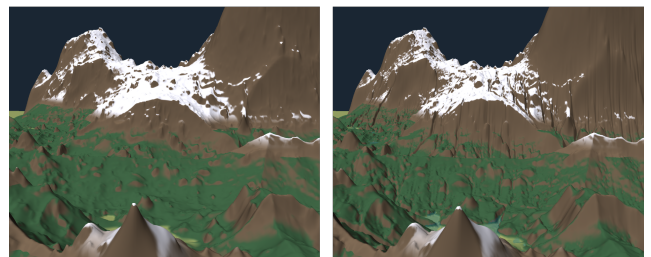


FIGURE 2 – Comparaison entre une génération procédurale classique par bruit de Perlin (gauche) et un terrain soumis aux forces d'érosion (droite).

Pour insuffler ce réalisme, la recherche s'est orientée vers la simulation des processus d'érosion, posant un défi computationnel de taille.

2.1 Érosion Hydraulique : Le Paradigme Eulérien

En 2007, Mei, Decaudin et Hu [1] ont démontré qu'il était possible de simuler l'érosion hydraulique à des fréquences d'affichage interactives. Au lieu de s'engager dans la résolution tridimensionnelle extrêmement lourde des équations de Navier-Stokes, leur modèle exploite une simplification bidimensionnelle : les équations en eaux peu profondes (*Shallow Water Equations*).

Leur approche s'inscrit dans un paradigme **Eulérien** : l'espace est modélisé sous la forme d'une grille discrète et immuable. Chaque cellule de cette matrice maintient et met à jour un état physique local (hauteur de la roche, hauteur de la colonne d'eau, vitesse du flux, quantité de sédiments). L'écoulement hydrique y est évalué via le "modèle des tuyaux virtuels" (*Virtual Pipe Model*), qui calcule l'accélération de l'eau générée par la différence de pression hydrostatique entre cellules adjacentes. Le transport de la masse sédimentaire est ensuite résolu par advection semi-Lagrangienne rétrograde, garantissant une stabilité inconditionnelle face aux problèmes de divergence numérique.

2.2 Le Débat Conceptuel : Grilles vs Particules

L'état de l'art oppose fréquemment ce paradigme Eulérien aux approches **Lagrangiennes** (systèmes de particules ou *droplet-based erosion*), récemment perfectionnées par Hartley et al. [4]. Bien que la méthode particulaire excelle visuellement dans le tracé organique de micro-incisions et de ravines abruptes, elle se heurte à des limites matérielles sévères lors de son exécution sur des architectures massivement parallèles (GPU), notamment en raison des conditions de course (*Race Conditions*).

Dans un modèle Lagrangien, la gravité et la topographie forcent naturellement des milliers de particules virtuelles à converger vers les mêmes vallées ou lits de rivières. Sur le plan matériel, cela se traduit par de multiples *threads* concurrents tentant de modifier simultanément la même adresse mémoire (la hauteur d'une cellule spécifique).

Afin d'éviter la corruption des données, la résolution de ces conflits exige l'utilisation d'opérations atomiques (*Atomic Operations* ou *InterlockedAdd*). Cependant, cette contrainte sérialise localement l'exécution, provoque des conflits d'accès au cache L2 (*cache thrashing*), et anéantit les bénéfices de performance offerts par le parallélisme massif du GPU.

À l'inverse, l'approche Eulérienne, bien que soumise à des artefacts d'advection sur les pentes raides, garantit un schéma d'accès mémoire déterministe et spatialement cohérent (un *thread* = une cellule lue/écrite). Cette garantie de performance et de stabilité justifie pleinement le choix de ce socle algorithmique pour notre implémentation temps réel.

2.3 La Révolution Matérielle des Compute Shaders

L'implémentation originelle de Mei [1] était bridée par les limitations matérielles de son époque. L'utilisation contrainte des Fragment Shaders imposait un détournement lourd du pipeline de rasterisation : nécessité de générer de la géométrie fictive, packaging artificiel des données physiques dans les canaux RGBA, et redondance désastreuse des accès à la mémoire vidéo globale (*VRAM*).

Notre projet tire pleinement parti de la transition architecturale vers les **Compute Shaders** (introduits avec OpenGL 4.3). Cette technologie s'affranchit du pipeline graphique traditionnel. Les données sont structurées dans des textures ou des *Shader Storage Buffer Objects* (SSBO) accessibles directement en lecture et écriture (*imageLoad/imageStore*). Bien que la technique du double tamponnage (*ping-pong*) reste indispensable pour figer l'état t lors du calcul de l'état $t + 1$, les Compute Shaders permettent l'exploitation de la Mémoire Partagée locale (*Shared Memory*) au sein des *Workgroups*. Ce cache ultra-rapide limite les requêtes redondantes vers la VRAM, déplaçant le goulot d'étranglement de la bande passante vers la capacité de calcul pur de la puce.

2.4 L'Érosion Thermique comme Nécessité Topologique

Si le modèle hydraulique creuse des lits de rivières convaincants, il engendre un artefact visuel majeur : l'apparition de falaises parfaitement verticales au bord des flux d'eau, des structures géologiquement instables. Schott et al. [3] soulignent que ces approches manquent de précision et requièrent des processus d'amplification.

Pour répondre à cette problématique, Jákó et Szirmay-Kalos [2] ont mis en évidence la nécessité de coupler l'hydraulique à un modèle d'**érosion thermique**. Ce processus physique complémentaire simule l'effondrement gravitationnel, le détachement de la roche et la formation d'éboulis dès lors que l'inclinaison locale dépasse l'angle de repos spécifique au matériau.

3 Méthodologie et Ingénierie

3.1 Architecture Logicielle et Outils

Pour garantir des performances optimales et un déploiement robuste, le cœur de notre application a été développé en C++ standard, couplé à l'API graphique **OpenGL** (version 4.3 minimum requise pour le support natif des Compute Shaders). L'interface utilisateur, essentielle pour transformer ce projet académique en un véritable outil de conception, a été intégrée via la bibliothèque **Dear ImGui**. Cette pile technologique nous offre un contrôle bas niveau sur le matériel tout en facilitant le débogage interactif.

3.2 Génération Topographique et Importation

Le socle rocheux sur lequel opère notre simulation peut être défini de deux manières distinctes, sélectionnables dynamiquement via l'interface utilisateur lors de l'initialisation.

La première approche repose sur une génération procédurale exécutée intégralement sur le GPU. Pour sculpter ces massifs virtuels, nous combinons plusieurs octaves de bruit pseudo-aléatoire : un bruit de crêtes (*Ridged Noise*) est employé pour générer des chaînes de montagnes acérées, tandis qu'un mouvement brownien fractionnaire (FBM - *Fractional Brownian Motion*) apporte les vallonnements organiques du terrain de base.

La seconde approche intègre la prise en charge de cartes de hauteurs réelles (*heightmaps*) importées sous forme d'images en niveaux de gris (ex : données topographiques de la NASA). L'implémentation de cette fonctionnalité s'est révélée indispensable pour constituer notre référentiel d'évaluation. Elle permet de mener des comparaisons rigoureuses et reproductibles entre nos différentes optimisations algorithmiques (ainsi qu'avec l'implémentation CPU de référence) sur une topologie d'entrée strictement identique.

3.3 Structures de Données GPU

La simulation s'appuie sur une grille régulière bidimensionnelle (généralement 1024×1024). Afin de maintenir l'état du système hydrodynamique, nous allouons plusieurs textures OpenGL avec un typage interne de haute précision (32-bit flottant). Les variables fondamentales suivantes sont stockées pour chaque cellule (x, y) [1] :

- La hauteur du terrain b et la hauteur d'eau d (Textures `GL_R32F`).
- La quantité de sédiments en suspension s (`GL_R32F`).
- Le vecteur de flux sortant $f = (f^L, f^R, f^T, f^B)$ vers les 4 directions cardinales (Texture `GL_RGBA32F`).
- Le vecteur vitesse du fluide $\vec{v} = (u, v)$ (Texture `GL_RG32F`).

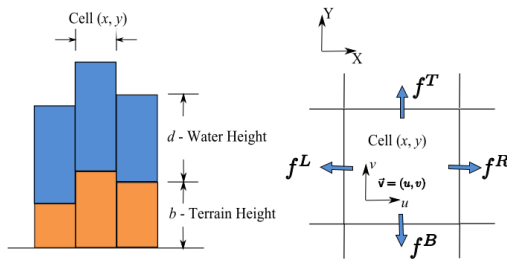


FIGURE 3 – Représentation spatiale des variables d'état dans une cellule de la grille (d'après Mei et al. [1]).

Pour le calcul des écoulements sans recourir à des équations différentielles instables, le système emploie le modèle des tuyaux virtuels (*Virtual Pipe Model*). La variation du flux vers une cellule voisine est déterminée par la gravité g , la largeur du tuyau A , sa longueur l et la différence de

hauteur totale $\Delta h = (b_{local} + d_{local}) - (b_{voisin} + d_{voisin})$:

$$f_{t+\Delta t} = \max \left(0, f_t + \Delta t \cdot A \cdot \frac{g \cdot \Delta h}{l} \right)$$

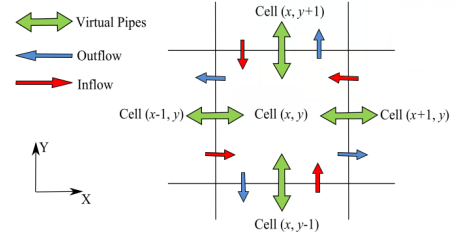


FIGURE 4 – Le modèle des tuyaux virtuels gérant les échanges de flux bidimensionnels (d'après Mei et al. [1]).

Pour éviter l'apparition aberrante de hauteurs d'eau négatives (si la somme des flux sortants excède le volume d'eau réel de la cellule), un facteur de mise à l'échelle scalaire K est calculé localement et appliqué uniformément aux quatre composantes du flux, garantissant la conservation stricte de la masse hydrique.

3.4 Améliorations du Modèle de Mei : Apports de Jákó et Szirmay-Kalos

Bien que le modèle de Mei et al. [1] pose des fondations algorithmiques extrêmement solides pour la simulation Eulérienne en temps réel, son exécution sur des échelles de temps géologiques révèle certaines limites physiques inhérentes : creusement infini des fonds sous-marins, arrachement sédimentaire isotrope, et légères instabilités de la surface hydrique.

Afin de pallier ces défauts et d'élever le réalisme de notre moteur, nous avons outrepassé l'implémentation classique en y intégrant trois corrections physiques majeures issues des travaux postérieurs de Jákó et Szirmay-Kalos [2]. Ces ajouts, intégrés directement au cœur de notre passe d'érosion et de sédimentation (Passe 4), modifient fondamentalement le comportement du fluide :

- **Inhibition de l'érosion en eaux profondes** (l_{max}) : Dans le modèle de 2007, un fleuve se jetant dans un lac continue d'éroder le fond marin de manière perpétuelle. Nous avons implémenté un facteur d'atténuation dynamique contraint par la profondeur locale. Lorsque la hauteur de la colonne d'eau dépasse un seuil critique, la capacité de transport s'annule. Ce mécanisme protège naturellement les fonds sous-marins et force le dépôt des sédiments aux embouchures, générant des deltas fluviaux et des hauts-fonds organiques.
- **Collision Hydrologique 3D** : La capacité d'arrachement de la roche ne dépend plus uniquement d'une simple pente bidimensionnelle. L'algorithme calcule désormais la force d'impact via le produit scalaire entre le vecteur de vitesse tridimensionnel du fluide et la normale de la surface heurtée. Une masse

d'eau percutant un obstacle géométrique de plein fouet creusera ainsi de manière exponentiellement plus agressive qu'un fluide glissant parallèlement à une pente douce.

- **Couplage Volumétrique Eau-Sédiment** : L'approche originelle omettait le volume physique occupé par la terre en suspension dans l'eau. Nous avons résolu cette incohérence par un couplage strict des états : lorsqu'un volume de roche est arraché, il s'ajoute instantanément à la colonne d'eau locale. Inversement, la déposition sédimentaire abaisse le niveau d'eau. Cette correction de la conservation des masses neutralise les comportements indésirables, garantissant la stabilité absolue de la surface des lacs sur de très longues sessions de simulation.

L'intégration de ces contraintes transforme le solveur mathématique de base en une véritable simulation géomorphologique, et ce, avec un coût computationnel supplémentaire rendu totalement invisible par l'architecture des Compute Shaders.

3.5 Érosion Thermique Avancée : Dureté Locale et Voisinage de Moore

La modélisation de l'érosion thermique sur une grille discrète soulève généralement deux problématiques géométriques majeures : l'apparition de structures pyramidales artificielles (lorsque l'évaluation se restreint à un voisinage de von Neumann à 4 directions) et un aspect "plastique" globalisé (causé par l'utilisation d'un angle de talus constant sur l'ensemble du domaine).

Pour contourner ces défauts, notre algorithme intègre directement le modèle avancé de Jákó et Szirmay-Kalos [2]. L'instabilité mécanique et le transfert de sédiments y sont évalués sur un voisinage de Moore (8 directions), permettant la formation de cônes d'éboulis organiques et parfaitement circulaires. Plutôt que de déplacer la matière de manière séquentielle et gloutonne, nous avons appliqué le paradigme Eulérien des "tuyaux virtuels" au substrat rocheux lui-même.

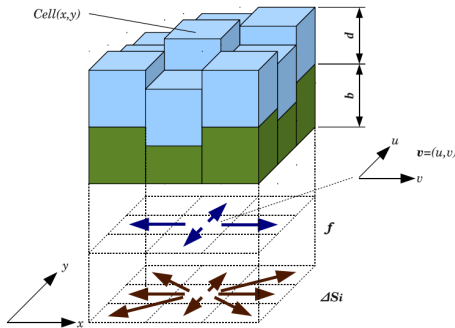


FIGURE 5 – Comparaison géométrique du modèle des tuyaux virtuels appliqué à l'écoulement d'eau (voisinage de von Neumann à 4 directions, flèche bleue) et à l'éboulement thermique des sédiments (voisinage de Moore à 8 directions, flèche rouge).

Une première passe calcule les volumes de terre devant s'effondrer et les stocke sous forme de flux directionnels virtuels, tandis qu'une seconde passe synchronise la nouvelle topologie en additionnant ces flux. Cette architecture en deux temps résout élégamment les dépendances d'écriture tout en garantissant la conservation stricte du volume topologique.

De plus, afin de briser l'homogénéité visuelle typique des simulations standards (l'effet plastique), la tangente de l'angle de talus est modulée en chaque point par un coefficient de dureté locale $R(x, y)$. Ce coefficient est généré dynamiquement sur le GPU en combinant l'élévation (la roche en haute altitude étant physiquement plus dense) et un bruit pseudo-aléatoire simulant l'hétérogénéité des strates géologiques. Les sommets peuvent ainsi soutenir des falaises presque verticales, tandis que les vallées s'affaissent en pentes douces, créant une richesse géomorphologique inatteignable avec un angle de repos statique.

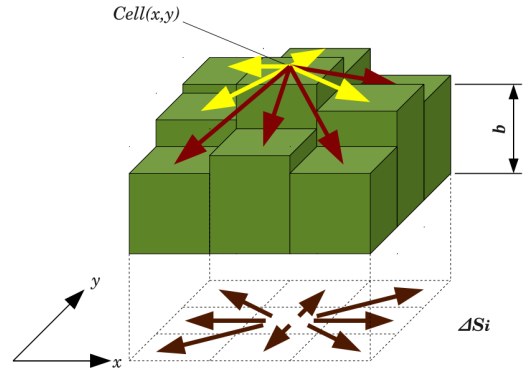


FIGURE 6 – Mécanisme des tuyaux virtuels appliqué à l'érosion thermique. Les transferts de sédiments (flèches rouges) s'effectuent exclusivement vers les cellules voisines dont le dénivelé excède l'angle de talus local. À l'inverse, l'éboulement est mathématiquement inhibé (flèches jaunes) vers les zones considérées comme géologiquement stables.

3.6 Stabilité Numérique et Condition CFL

Un défi inhérent aux solveurs de fluides explicites réside dans leur instabilité si le pas de temps d'intégration (Δt) est inadapté à la vitesse maximale du fluide. Si l'eau se déplace sur une distance supérieure à la taille d'une cellule en une seule itération, le modèle mathématique s'effondre. Pour éviter ces explosions numériques, nous avons implémenté une vérification stricte de la condition de Courant-Friedrichs-Lewy (CFL) :

$$\Delta t \leq \frac{\Delta x}{v_{max}}$$

Notre interface ImGui avertit l'utilisateur de manière dynamique si les paramètres de gravité ou d'apport d'eau nécessitent une réduction du pas de temps pour maintenir la cohésion mathématique de la simulation.

3.7 Implémentation du Pipeline et Gestion Mémoire

La boucle de calcul est exécutée par des groupes de travail (*Workgroups*) matériels de 16×16 *threads*, une taille empiriquement optimale pour maximiser l'occupation des cœurs de calcul de l'architecture graphique.

Pour pallier les problèmes de concurrence en écriture inévitables sur une grille globale, notre architecture repose sur un **double tamponnage** (*ping-pong buffering*). Les textures stockant les hauteurs, l'eau et les sédiments sont dupliquées. À l'itération i , les shaders lisent l'état figé dans les tampons **READ** et inscrivent les résultats dans les tampons **WRITE**.

L'intégrité de la simulation est assurée par l'injection de barrières de mémoire (`glMemoryBarrier(GL_SHADER_IMAGE_ACCESS_BARRIER_BIT)`) entre chaque appel de noyau. Cette instruction force le GPU à purger les données de son cache L1/L2 vers la mémoire globale avant d'autoriser le shader suivant à débiter. Grâce à ce séquençage en "aller-retour" (du tampon **READ** vers **WRITE**, puis inversement), l'état physique final est toujours garanti d'être valide et disponible dans le tampon **READ** pour le rendu visuel de la trame courante.

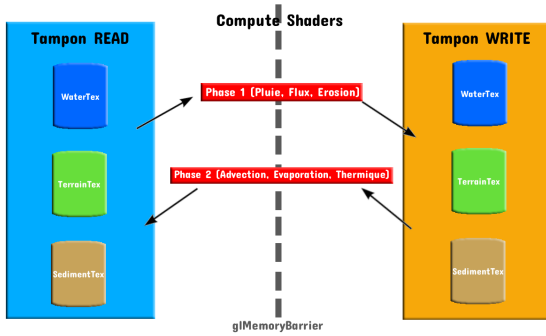


FIGURE 7 – Architecture du double tamponnage : les passes physiques écrivent dans le tampon temporaire (WRITE), tandis que les passes de post-traitement rapatrient l'état final dans le tampon principal (READ), sous la protection des barrières mémoire.

4 Détails de l'Implémentation du Pipeline

Notre pipeline d'érosion a été segmenté en huit passes de Compute Shaders distinctes, formant une boucle itérative où chaque étape dépend des résultats de la précédente. Pour pallier les manques du modèle historique, nous avons enrichi chaque phase par des modèles physiques plus récents.

1. Incrémentation du niveau d'eau (`water_increment.glsl`) : Cette passe initialise l'apport d'eau dans le système. La hauteur d'eau locale d est mise à jour selon l'équation de précipitation : $d_{t+\Delta t} = d_t + (\Delta t \cdot r)$, où r est le taux de pluie. L'application d'une pluie uniforme

est cruciale pour la stabilité numérique, car elle évite les zones de vide hydrique qui pourraient générer des gradients de pression erronés. Nous y intégrons également les sources de rivières permanentes par une injection localisée de débit.

2. Calcul des Flux par Tuyaux Virtuels (`flux_compute.glsl`) : Le cœur du solveur de fluide repose sur le modèle des tuyaux virtuels de Mei et al. [1]. Chaque cellule est connectée à ses voisines par des conduits imaginaires. Le flux sortant f vers une direction donnée est proportionnel à la différence de pression hydrostatique Δh (différence entre les hauteurs totales $b + d$ des deux cellules). L'accélération du flux est régie par :

$$f_{t+\Delta t} = \max \left(0, f_t + \Delta t \cdot A \frac{g \cdot \Delta h}{l} \right)$$

où A est la largeur du tuyau et l sa longueur. Un facteur de mise à l'échelle K est calculé pour s'assurer que la somme des flux sortants ne dépasse jamais le volume d'eau disponible dans la cellule, garantissant ainsi la positivité de la hauteur d'eau.

3. Mise à Jour et Champ de Vitesse (`water_update.glsl`) : La nouvelle hauteur d'eau d_{t+1} est calculée par le bilan des flux entrants et sortants. Parallèlement, le vecteur vitesse $\vec{v} = (u, v)$ du fluide est inféré en calculant le débit net traversant la cellule sur les axes X et Y . Nous y avons introduit un terme de friction qui dissipe l'énergie cinétique sur les pentes abruptes, empêchant l'eau de gagner une vitesse infinie et permettant un temps de résidence suffisant pour que l'érosion s'opère.

4. Érosion et Sédimentation Avancées (`erosion_deposition.glsl`) : Cette passe calcule le transfert de masse entre le substrat rocheux b et les sédiments en suspension s . Elle repose sur la capacité de transport C , qui définit la quantité maximale de terre que l'eau peut porter. Suivant les concepts introduits précédemment (cf. Section 3.4), nous calculons C en intégrant l'impact 3D (produit scalaire) et l'atténuation en eaux profondes :

$$C = K_c \cdot \max(0.15, -\vec{v}_{3D} \cdot \vec{n}) \cdot |\vec{v}| \cdot l_{max}$$

Si $C > s$, l'eau arrache de la roche ($b \leftarrow b - \Delta s$) ; si $C < s$, elle dépose le surplus ($b \leftarrow b + \Delta s$). Le volume de la colonne d'eau est ajusté simultanément en mémoire (*couplage volumétrique*) pour respecter la conservation de masse du système.

5. Transport Sédimentaire par Advection (`sediment_transport.glsl`) : Les sédiments s ne restent pas immobiles ; ils sont portés par le courant. Nous résolvons l'équation d'advection par une méthode semi-Lagrangienne : pour chaque cellule, nous cherchons la position d'origine de la particule à $t - \Delta t$ en remontant le

vecteur vitesse $\vec{v}$. La valeur est récupérée par interpolation bilinéaire. Pour stabiliser la simulation face aux erreurs de divergence numérique, un facteur de micro-atténuation est appliqué.

6. Évaporation et Conditions aux Limites (evaporation.gls1) : L'eau s'évapore selon un taux K_e . Nous avons implémenté une évaporation différentielle : l'eau stagnante (vitesse nulle) s'évapore plus vite que l'eau en mouvement, favorisant le séchage des flaques isolées. Cette passe gère aussi les conditions de sortie : aux bords de la carte, la hauteur d'eau est forcée à zéro, simulant un drain vers un océan infini.

7. Calcul des Flux Thermiques (thermal_flux.gls1) : Indépendamment de l'eau, cette passe simule l'instabilité mécanique des pentes selon le modèle thermique détaillé en Section 3.5. L'algorithme calcule l'excès de pente sur un voisinage de Moore et génère la carte de dureté locale $R(x, y)$ via un bruit spatial. Le volume d'éboulement est alors distribué proportionnellement dans 8 directions virtuelles. L'érosion thermique est localement inhibée par la présence d'eau pour protéger les ravins hydriques.

8. Application Thermique (thermal_apply.gls1) : Le terrain est mis à jour en additionnant les flux thermiques entrants et sortants calculés à la passe précédente. Cette séparation en deux passes évite les conditions de course (*race conditions*) en mémoire partagée, garantissant une conservation parfaite de la terre effondrée.

4.1 Limites Numériques et Conditions aux Frontières

La transposition d'un modèle mathématique continu sur une grille discrète engendre des instabilités inhérentes aux méthodes de résolution choisies. Bien que notre architecture garantisse certaines lois physiques, deux limites d'ingénierie ont nécessité des ajustements pragmatiques.

Le premier défi concerne la **non-conservation de la masse sédimentaire**. Si le volume d'eau est strictement conservé par le modèle des tuyaux virtuels (via le facteur d'échelle K de la Passe 2), le transport des sédiments (Passe 5) s'appuie sur une advection semi-Lagrangienne rétrograde. Bien que cette méthode soit plébiscitée pour sa stabilité inconditionnelle face aux vitesses élevées du fluide, elle n'est mathématiquement pas conservative. Dans les zones où le champ de vélocité diverge (ex : le sommet d'une colline), de multiples cellules adjacentes peuvent "regarder en arrière" (*backtracing*) et échantillonner bilinéairement la même cellule source. Ce phénomène clone littéralement la charge sédimentaire. Sur un temps d'exécution prolongé, cette création de matière anti-physique se traduisait par des montagnes gagnant artificiellement en altitude.

Le second défi est inhérent à toute simulation sur grille finie : la gestion des **conditions aux limites**. Initialement

traitée comme un système fermé, la grille empêchait la matière de s'échapper. L'eau et les sédiments s'accumulaient inexorablement contre les frontières, élevant des barrages artificiels qui finissaient par noyer la topologie.

Pour restaurer l'intégrité de la simulation sans recourir à des solveurs strictement conservatifs complexes (dont le coût ruinerait nos performances GPU), nous avons mis en œuvre deux solutions intégrées aux passes 5 et 6 :

- **Micro-atténuation sédimentaire :** Afin de contrer la duplication numérique lors de l'advection, nous avons introduit un facteur d'atténuation infime. L'application d'un lessivage systématique (multiplication de la charge sédimentaire par 0,999 par itération) permet de compenser statistiquement la création artificielle de matière, stabilisant l'élévation globale du terrain sans perturber la dynamique de dépôt.
- **Système ouvert (Drains frontaliers) :** Pour dissiper l'accumulation aux frontières, nous avons imposé des conditions de Dirichlet nulles sur le périmètre extérieur. Les cellules bordant la carte agissent comme des "drains" continus où la hauteur d'eau et les sédiments sont forcés à zéro. Cette modification permet à la matière de s'évacuer définitivement, simulant l'écoulement naturel d'un bassin versant vers un océan virtuel infini.

5 Résultats et Évaluation

Afin de quantifier de manière irréfutable les bénéfices de l'architecture Compute Shader, nous avons développé une implémentation rigoureusement équivalente s'exécutant de manière séquentielle sur le processeur (CPU). Ce programme en C++ natif nous sert de *baseline* absolue pour mesurer le ratio d'accélération sur des *heightmaps* d'entrée identiques.

Notre évaluation intègre les mesures documentées par Mei et al. en 2007 [1] (GeForce 8800 GTX), ainsi que les résultats plus récents de Jákó et Szirmay-Kalos en 2011 [2] obtenus sur une architecture ATI Radeon HD4870. À titre de comparaison, le modèle de 2011 atteignait un temps de cycle global de 15,47 ms pour une grille de résolution similaire (1024×1014) [2]. Nos propres mesures ont été isolées via les *Timestamp Queries* d'OpenGL sur une plateforme composée d'un processeur AMD Ryzen 5 2600 et d'une carte graphique AMD Radeon RX 5700.

Étape de calcul	Mei (2007)	Jákó (2011)	Notre CPU	Notre GPU
Sim. Physique (Eau)	6.65 ms	N/A	898.244 ms	0.408 ms
Érosion Thermique	N/A	N/A	821.458 ms	0.223 ms
Rendu Visuel	10.31 ms	N/A	3.227 ms	0.407 ms
Temps de Cycle	16.95 ms	15.47 ms	1722.929 ms	1.037 ms

TABLE 1 – Tableau comparatif des performances (Résolution : 1024×1024). Les données historiques [1, 2] servent de référentiels. Un cycle de 1.037 ms correspond à une fréquence théorique de 964 images par seconde (FPS).

L'analyse de ces métriques illustre un saut technologique majeur. L'implémentation CPU est terrassée par la com-

plexité algorithmique du voisinage de Moore (8 directions) et les calculs de collision 3D, aboutissant à un temps de cycle critique de 1,72 seconde. Ce délai rend toute interaction impossible sur processeur (0,58 FPS).

Le basculement sur GPU via Compute Shaders pulvérise ces limitations, ramenant la boucle complète à seulement 1,037 ms. Le facteur d'accélération global entre notre implémentation CPU et GPU atteint ainsi le ratio colossal de **1661x**. L'optimisation est particulièrement visible sur l'érosion thermique : malgré la complexité du modèle de Jákó (calcul de flux directionnels et dureté locale), le GPU exécute cette tâche en 0,223 ms (soit une accélération de 3680x face au CPU). Ces résultats prouvent que l'architecture moderne permet d'exécuter des modèles physiques autrefois coûteux ([1, 2]) en une fraction de milliseconde.

5.1 Interactivité et Découplage Temporel

Cette colossale réserve de puissance (framerate théorique frôlant les 960 FPS) nous a permis d'implémenter un découplage temporel profond entre le moteur physique et le moteur de rendu. L'utilisateur peut, via l'interface ImGui, ordonner l'exécution de multiples itérations physiques (`ITERATIONS_PER_FRAME`, paramétrable jusqu'à 10) pour chaque image affichée à l'écran.

Afin de donner une échelle de grandeur à cette performance, nous avons introduit un facteur de conversion vers le temps géologique. En considérant qu'une seconde de temps physique simulé équivaut à 10 000 ans d'érosion naturelle, notre simulateur permet de compresser les millénaires de manière fulgurante. Avec un pas de temps $\Delta t = 0,0015$, chaque itération fait avancer l'érosion de 15 ans. À son paramétrage maximal de 10 itérations par trame, l'application calcule 150 années géologiques par image. Ainsi, une seule seconde de simulation en temps réel, standardisée à 60 FPS, permet d'écouler **9 000 années géologiques** (un chiffre qui s'envole à plus de 144 000 années par seconde si le framerate est débridé).

Ce mécanisme permet d'observer en quelques minutes des processus qui nécessiteraient normalement des millions d'années, comme le creusement de canyons profonds ou l'aplanissement de massifs montagneux. L'interface offre de surcroît la possibilité d'altérer les constantes physiques ou de déplacer des sources de rivières à la volée, transformant le programme en un puissant laboratoire géomorphologique interactif où l'utilisateur navigue fluidement à travers les époques.

6 Conclusion et Perspectives

L'objectif central de ce projet TER consistait à concevoir un moteur d'érosion procédurale interactif, s'appuyant sur le modèle historique de Mei et al. [1], tout en comblant son manque de réalisme topologique [3] via l'intégration des technologies graphiques contemporaines. Ce cahier des charges a été rempli avec succès.

Sur le plan géométrique, l'intégration du modèle de Jákó et Szirmay-Kalos [2] s'est révélée déterminante. L'ajout

du frein en eaux profondes, de l'érosion par collision 3D, et de l'érosion thermique gérée par tuyaux virtuels sur 8 directions a permis d'obtenir une topologie extrêmement organique, riche en deltas et en éboulis hétérogènes. Sur le plan de l'ingénierie logicielle, la structuration stricte des données en *ping-pong buffering*, la conservation locale de la masse (incluant le couplage du volume Eau-Sédiment), et la maîtrise des barrières mémoire au sein des Compute Shaders ont abouti à des performances de premier ordre (1,037 ms par cycle pour plus d'un million de sommets). L'analyse croisée avec notre propre version CPU de référence a permis de quantifier scientifiquement l'apport matériel, validant une accélération matérielle excédant 1660x.

Fort de cette architecture stable et massivement optimisée, les perspectives d'évolution sont particulièrement prometteuses. La contrainte inhérente aux *heightmaps* 2.5D (l'impossibilité de modéliser des surplombs, grottes ou arches rocheuses) pourrait être levée en orientant le moteur vers des structures stratifiées volumétriques (*Multi-Layered Heightmaps*) [5]. À une échelle écosystémique, le couplage de cette érosion avec un modèle végétal dynamique où l'humidité favorise la croissance d'une flore dont le réseau racinaire renforce mécaniquement la résistance du sol à l'érosion hydrique constituerait le défi ultime pour la génération procédurale de biomes planétaires complets.

Références

- [1] X. Mei, P. Decaudin, and B. Hu. *Fast hydraulic erosion simulation and visualization on gpu*. In 15th Pacific Conference on Computer Graphics and Applications (PG'07), pages 47-56. IEEE, 2007.
- [2] B. Jákó and L. Szirmay-Kalos. *Fast Hydraulic and Thermal Erosion on the GPU*. In Eurographics (Short Papers), pages 57-60, 2011.
- [3] H. Schott, E. Galin, E. Guérin, A. Paris, and A. Peytavie. *Terrain amplification using multi scale erosion*. ACM Transactions on Graphics (TOG), 43(4) :1-12, 2024.
- [4] M. Hartley, N. Mellado, C. Fiorio, and N. Faraj. *Flexible terrain erosion*. Computer Graphics Forum, 43(2), 2024.
- [5] S. Nilles, J. Günther, M. Wagner, and C. Müller. *3D Real-Time Hydraulic Erosion Simulation using Multi-Layered Heightmaps*. In Vision, Modeling, and Visualization (VMV), 2024.